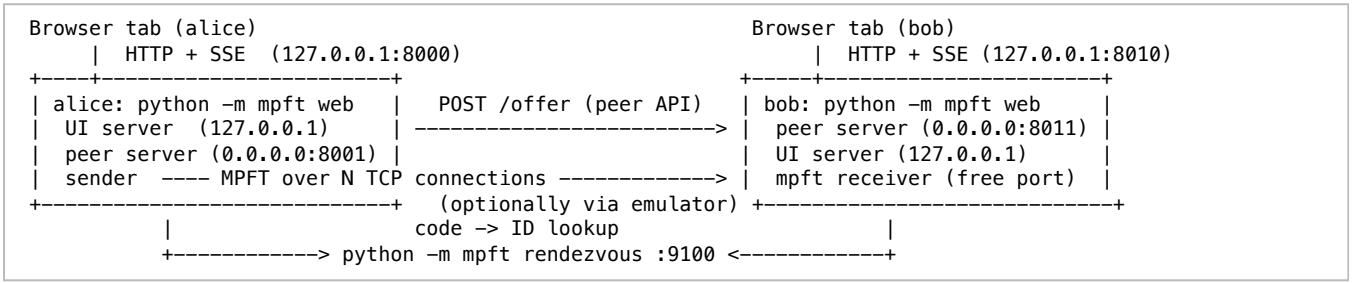
Web interface

A browser front end for sending and receiving files, with a live view of what the protocol is doing. The data plane is unchanged: the sender app calls `mpft.sender.run_transfer`, the receiver app runs the ordinary `mpft.receiver`, so Wireshark still sees plain MPFT.

Architecture

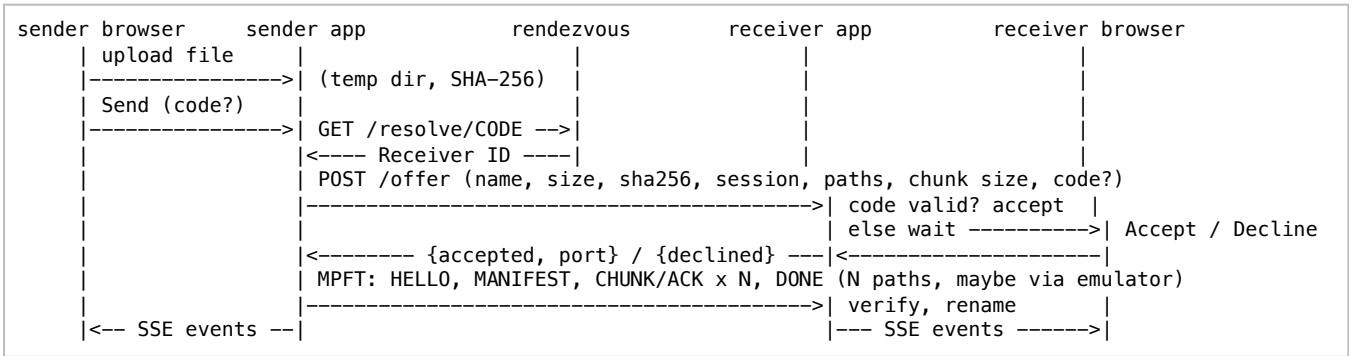


Piece	File	Job
Event bus	`mpft/events.py`	Thread-safe; sender, receiver and emulator emit, the web layer subscribes
Node	`web/node.py`	Identity, one-time code, offers, received files, history
Peer API	`web/peer.py`	Only `GET /health` and `POST /offer`, bound to 0.0.0.0
UI server	`web/ui.py`, `web/feed.py`	Pages, JSON API, SSE stream, downloads. Bound to 127.0.0.1
Send job	`web/sendjob.py`	Resolve target, offer, then `run_transfer`
Rendezvous	`web/rendezvous.py`	Code to Receiver ID lookup
Profiles	`web/netprofiles.py`	Emulator paths for eight scenarios (see Scenarios below)

Receiver ID. `base32("host:peer_port")` shown in groups of four letters. It decodes locally, so no server is needed. The host is the LAN IP, else 127.0.0.1 (`--host` overrides).

One-time code. Six characters without confusable letters, valid 10 minutes, single use. The receiver registers `code -> ID` with the rendezvous service; the sender resolves it. A valid code makes the receiver accept without a click.

Offer flow



Offers without a valid code wait up to 120 s for Accept or Decline. Received files are staged in `<out>/.incoming/<id>/`, then renamed into `<out>/` with a numeric suffix if the name exists (never overwritten).

Run on one Mac (two instances, two tabs)

```
source .venv/bin/activate
python -m mpft rendezvous --host 127.0.0.1 --port 9100                          # terminal 1
python -m mpft web --name alice --ui-port 8000 --peer-port 8001 --out ./alice-dl --host 127.0.0.1 --rendezvous
python -m mpft web --name bob   --ui-port 8010 --peer-port 8011 --out ./bob-dl   --host 127.0.0.1 --rendezvous
```

Open `http://127.0.0.1:8010/#/receive` (bob) and `http://127.0.0.1:8000/#/send` (alice). Copy bob's ID or code into alice's Send page, choose a file and press Send. With an ID, press Accept on bob's page; with a code there is no click.

Rendezvous (optional, for codes)

Codes need a lookup service: `python -m mpft rendezvous --port 9100`, then start each instance with `--rendezvous http://127.0.0.1:9100` (or set the URL in Receive > Settings). Sending by Receiver ID needs no service.

Security: MPFT has no built-in encryption and the rendezvous service has no authentication. Use it only on a trusted network; do not expose the peer port or the rendezvous port to the open internet. The UI server only listens on 127.0.0.1 and rejects foreign Host or Origin headers, because it controls the machine.

Network conditions

The Send page offers None, Asymmetric (paths progressively slower), Lossy (about 5 % of chunks damaged) and Path failure (last path dies 3 s after it connects; use a file of 50 MB or more). The emulator proxies run inside the sender app, in front of the receiver. The Transfer page then shows Kill path N and Set corruption (percent of chunks; converted to the emulator's per-16 KiB-block probability). `python -m mpft emulate --control-port 9300` exposes the same control over HTTP: `GET /state`, `POST /kill/N`, `POST /corrupt/RATE`.

Wireshark for a web transfer

Ports are chosen per transfer, so the Transfer page prints the exact command and filter. Re-copy `wireshark/mpft.lua` into the Wireshark plugin folder: it now also recognises MPFT by its header on any port.

```
sudo tcpdump -i lo0 -w mpft.pcap 'tcp port 59216 or tcp port 59217'   # as shown on the Transfer page
```

Display filters: `mpft.type == 4` (chunks), `mpft.type == 2` (manifest), `mpft.type == 6` (NACKs).

Limits and honest notes

- The offer's size and SHA-256 are what the receiver shows; the data plane verifies against the manifest the sender sends.
- Corruption in the UI is emulated on the sender side of the emulator; real packet loss is not emulated (TCP hides it).
- The event feed keeps the last 100 000 events per process, so a page reload replays a transfer; very large transfers with tiny chunks lose their oldest events.

Troubleshooting

Symptom	Cause and fix
"Could not reach the receiver"	Wrong ID, receiver app not running, or firewall. Open `http://RECEIVER:PEERPORT/health` from the sender Mac.
"No rendezvous URL is set"	Add it in Receive > Settings, or start with `--rendezvous`; or send by ID.
"Code not found or expired"	Codes live 10 minutes and work once. Press New code on the receiver.
"Too many wrong codes"	Five failures per minute per client are allowed; wait a minute.
Port already in use	A previous instance is still running: `lsof -i :8001`.
ID shows 127.0.0.1	No LAN address found; start with `--host <your LAN IP>` if needed.
Packets look like plain TCP in Wireshark	Old copy of `mpft.lua`; reload the new one.

Shared two-sided dashboard

- **Receiver ID** carries every address of the receiver ("h1,h2:port", base32). The sender probes each with `GET /health` and spreads the paths round-robin over those that answer. Send > Test connection shows per address reachable/unreachable with the reason.
- **Peer API** is exactly `GET /health`, `GET /addresses`, `POST /offer`, `POST /events/{session_id}`. The accept reply carries a session token; `/events` needs it in `X-Session-Token`.
- **Event forwarding**: the sender batches its events every 0.5 s to `/events/{id}`; the reply carries the receiver's new events, so both browsers show one merged log. Each event is tagged `side` and a per-side sequence `sseq`. Timestamps are shifted with an NTP-style offset measured on the offer round trip (the receiver's wait for Accept cancels out). Order: corrected time, then side, then `sseq`. `t` is epoch seconds (offset-corrected); the UI shows time since session start.
- **Events**: `session_started`, `offer_sent/received/accepted/declined`, `manifest_sent/received`, `path_up/down/restored`, `backoff`, `timeout`, `chunk_state` (pending, inflight, done, retransmitted, failed; offset, size, path, attempt), `message` (HELLO, MANIFEST, NACK, ERROR, DONE always; CHUNK/ACK sampled), `nack`, `requeue` (count, reason), `hash_chunk_ok/bad`, `hash_file_verified/mismatch`, `throughput_tick`, `rtt_sample`, `session_done`. `status/<role>-<session>.json` is unchanged.
- **Persistence**: `.mpft-state/<name>/sessions/<session>.jsonl` on each side. History > Replay re-runs a session through the same reducer (play, pause, 1x/4x/16x, scrub); History > JSON exports it.
- **Data ports**: `--data-port 9000` makes the receiver listen on the first free port in 9000..9099 (0 = any free port); use it for fixed firewall rules.

- **Resume**: received data is staged by file hash. If the receiver or network goes away the sender fails with a clear message and the `.part` file is kept; sending the same file again resumes from the chunks already held.
- **Transfer page**: session header, progress, canvas chunk map (hatched in flight, solid done, outlined retransmitted, crossed failed, path-colour tick), sortable chunk table, paths table, incidents table with a plain sentence per failure, six graphs on a shared time axis with incident markers, verification, network controls (Kill, Restore, set delay, set corruption), protocol trace, event log with side filter, Wireshark commands.

Runbook (one machine, two instances)

1. Start two instances as in "Run" above (alice on 8000, bob on 8010).
2. Receiver (bob): Receive page, press Copy ID. Optionally also use the 6-character code (valid 10 minutes, once).
3. Sender (alice): Send page, pick a file, paste the ID (and code), Test connection, Send.
4. Without a code, bob presses Accept. Both open the Transfer page and see the same dashboard.
5. macOS may ask "accept incoming connections" the first time: Allow. If it was denied: System Settings > Network > Firewall > Options and allow the project's `.venv` Python.

Troubleshooting (addition)

Symptom	Cause and fix
Test connection: "No answer (timeout)"	Other network, Mac asleep, firewall dropping packets, or WiFi client isolation. Try another network.
"Connection refused"	Address reachable, app not listening: check the ID and that the receiver app runs on that peer port.
"No route to host"	The machines are on different networks.
Transfer failed, "send the same file again"	Receiver or network dropped; resend the same file and it resumes.
Settings > self-test says unreachable for a LAN address	macOS firewall blocks Python; allow it.

Honest security notes

No built-in encryption or authentication: chunks, events and offers are plain TCP and HTTP. Protections in the app: the UI server listens on 127.0.0.1 only and checks Host/Origin; the peer server exposes only four endpoints; `/events` needs a per-session token; codes are single-use, expire after 10 minutes and are rate-limited; names are sanitised, files never overwritten, 2 GB default limit; the receiver must press Accept unless a valid code was given.

Receiving and downloading files

- A verified file (whole-file SHA-256 checked, renamed from `.incoming`) is recorded in `<downloads>/.mpft-received.json` (`web/received.py`), so the list survives a restart. Names never overwrite: a second `payload.bin` is stored as `payload (1).bin`.

- A transfer that fails (sender gives up, hash mismatch) is also recorded, with `status: failed` and a message, and has no file and no Download button.
- API: `GET /api/files` (list), `GET /api/files/<id>/download` (attachment, HTTP Range supported, 404 for unknown ids, failed entries or a name that resolves outside the downloads dir), `DELETE /api/files/<id>`. Clients never send a path, only the id we generated.
- Receive page: table (name, size, SHA-256, received at, verified) with Download and Delete per row; it refreshes on the `file_received` event. The Transfer page of a finished session shows the same Download button in the header strip on the receiver's side.

Graphs

Six SVG graphs (throughput per path and total, chunks done, in-flight per path, RTT per path with min-max band, cumulative retransmissions, byte share) in `web/static/graphs.js`. Each SVG is drawn at its real pixel width and redrawn by a `ResizeObserver` (so a graph built in a hidden container is never 0 px wide), redraws are throttled to about 4 per second, and an empty graph says "Waiting for data". Dashed vertical markers: `path N down`, `requeued N`, `recovered`. Graphs are rebuilt from the stored events, so they render the same live, mid-transfer and after a reload.

Visual design and tests

Tokens (colours, spacing, fonts, path palette) live in `web/static/tokens.css` (light default, dark via `prefers-color-scheme` and the Theme toggle), layout in `layout.css`, components in `components.css`. Text contrast is checked by `tests/test_design_contrast.py` (4.5:1 text, 3:1 chart colours, both themes). Browser tests: `.venv/bin/pytest tests/e2e` starts two real instances (ports 8000/8001/9000 and 8010/8011/9010), sends a 30 MB file with the path-failure condition through alice's page, and asserts on console errors, SSE events, graphs, downloads (SHA-256 of the downloaded bytes), the failed-transfer case, theme, keyboard focus and phone-width layout. `MPFT_SHOTS=1` also writes the screenshots.

Sender view vs receiver view

Both sides open the same session, but the page is role-aware (`web/static/hero.js`, order table in `transfer.js`):

	Sender	**Receiver**
Page title	"Sending name"	"Receiving name"
Banner (left bar)	accent colour; "Delivered to bob" / "Not delivered" and a Send another file button	green bar; "Saved and verified" with a Download button / "Nothing was saved" and the reason
Panel order	summary, progress, chunk map, graphs, paths, network controls, incidents	verification first, then summary, progress, chunk map, incidents, graphs
Controls	Kill / Restore path, delay, corruption (the emulator runs on the sender)	none; it only observes and downloads

Scenarios

Chosen in the Send form (own Scenario panel, with a one-line description). Each runs through the emulator in front of the receiver.

Scenario	What happens	Result
Clean network	direct connections	success
Clean but slow	16 Mbit/s per path, 20 ms delay (long smooth curves)	success
Asymmetric	slower, further paths	success
Lossy	5% of chunks corrupted, NACK + retransmit	success
Heavy loss	20% of chunks corrupted	success, many retransmissions
Path failure	last path dies for good after 3 s	success on the remaining paths
Path flaps	one path dies at 2 s, returns 3 s later (backoff, reconnect)	success
Total failure	all paths die after 1 s (use a file over 2 MB)	fails; nothing saved, no download

Graph time axis

Throughput is computed in the browser from chunk completions, in buckets of `duration / 20` (1 s only for transfers over 20 s), so a 0.4 s transfer still draws a curve. Transfers under 2 s use a millisecond axis; longer ones use seconds. The x range is the real duration, not a fixed 5 s. Series start at 0. Screenshots: `docs/screenshots/scn-*.png`.